

aelion

Marine Lannes

Maintenance prédictive

Optimisation, éco-conception &
explicabilité

Le modèle tourne. Et maintenant ?

?

Peut-on faire mieux ?

Chercher de meilleurs hyperparamètres, dans un budget.

OPTUNA

?

Combien ça coûte ?

Mesurer l'énergie et le CO₂ de l'entraînement.

CODECARBON

?

Pourquoi cette décision ?

Comprendre quelles features pèsent, et vérifier le métier.

SHAP

01

Optimiser sans gaspiller



Optuna + éco-conception (CodeCarbon)

Tuner : régler ce qui ne s'apprend pas

Paramètres — appris

Les poids du modèle, ajustés automatiquement pendant l'entraînement.

Ex. : structure des arbres XGBoost.

Hyperparamètres — choisis

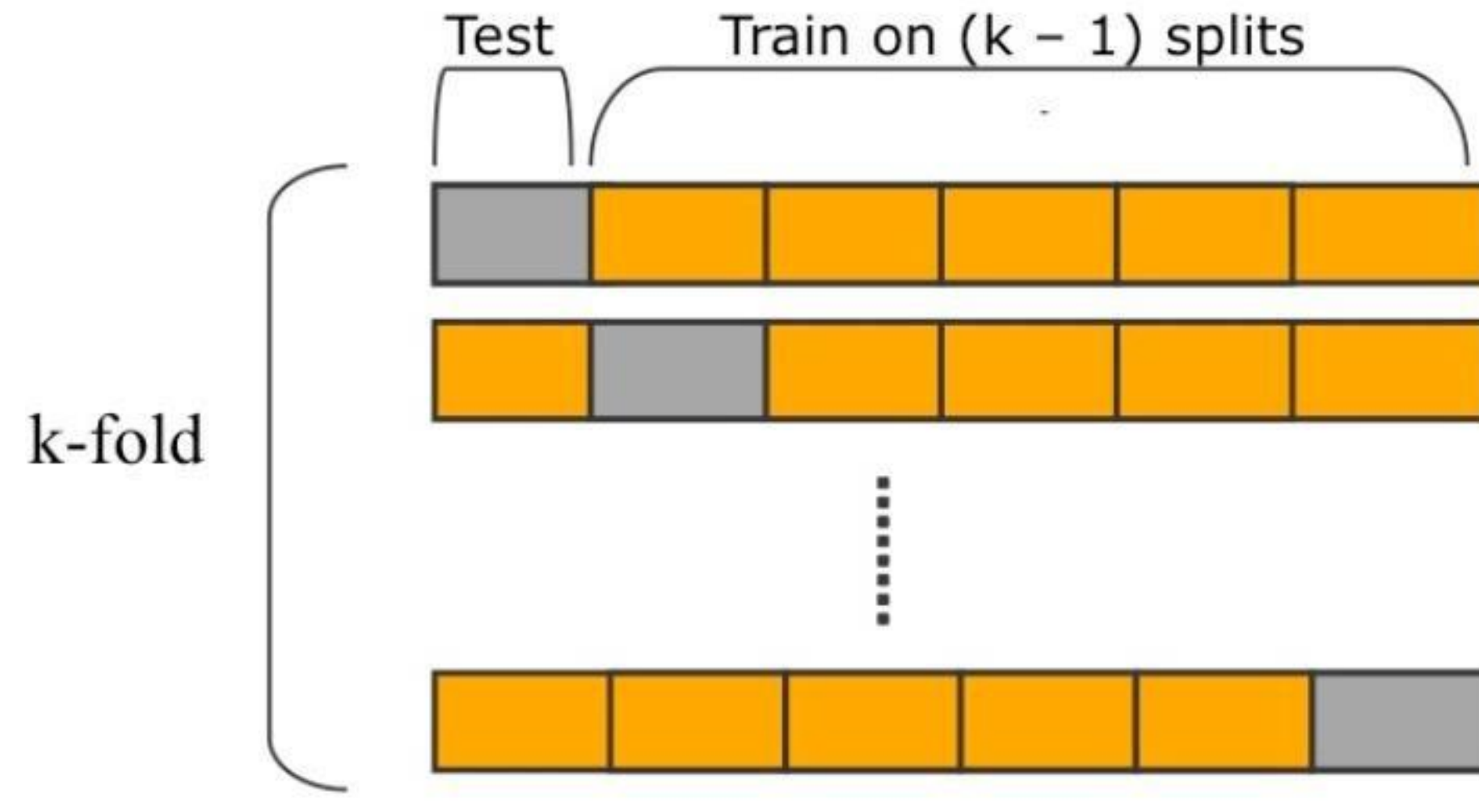
Profondeur, learning rate, nombre d'arbres... fixés AVANT l'entraînement.

C'est eux qu'on optimise.

Le tuning cherche la meilleure combinaison d'hyperparamètres — sur une surface de réponse bruitée, sans gradient.

À la main : lent et biaisé. Grille exhaustive : explosif. → il faut une recherche guidée.

Cross Validation



Résultat : K évaluations

→ Leur moyenne donne une évaluation plus fiable

→ Leur écart-type donne l'intervalle de confiance

Attention aux performances !

Grille exhaustive vs recherche guidée

Grille — GridSearchCV



- Teste toutes les combinaisons, en aveugle.
- Le coût explose en N dimensions.
- Limité aux points de la grille.

Recherche guidée — Optuna



- Apprend des essais passés (TPE).
- Budget borné + pruning.
- Explore l'espace continu.

Grille = inspecter chaque case de l'échiquier · Optuna = chasse au trésor (plus chaud / plus froid). RandomizedSearch est l'intermédiaire.

Study

La campagne d'optimisation complète.

Trial

Un essai :
un jeu d'HP
→ entraîne
→ score.

Sampler (TPE)

Propose le prochain essai à
partir de l'historique.

Pruner

Coupe tôt les essais non
prometteurs.

Boucle : chaque essai informe le suivant (TPE) ; les essais faibles sont coupés (pruner). On fixe un budget ($n_trials + timeout$) et une graine pour la reproductibilité.

Budget borné

n_trials + timeout. On s'arrête quand le gain ne vaut plus le calcul.

Critère clair

On choisit une métrique d'évaluation qui correspond à notre objectif.

Espace plausible

Plages resserrées : moins cher, optima plus robustes.

Reproductibilité

Graine fixée + étude stockée : on peut rejouer et auditer.

Piège — optimiser sur le test = sur-ajustement du tuning. On tune sur la validation ; le test reste intouché jusqu'à la fin.

Mesurer le coût : CodeCarbon

Ce qu'on mesure

Énergie consommée (kWh) et CO₂ émis (gCO₂eq).

Estimé via le mix électrique — FRA, bas carbone.

Un emissions.csv par run, traçable.

Bonnes pratiques green AI

Parcimonie : le modèle le plus simple qui suffit.

Early stopping + pruning des essais faibles.

Mutualiser et réutiliser les entraînements.

Le bon réflexe : raisonner en coût par point de performance gagné.

Arbitrer : perf × coût × carbone

Stratégie	AUC	Coût CO ₂	Verdict
Baseline	0,81	—	référence
Optuna frugal	0,84	faible	bon compromis
Optuna lourd	0,85	élevé	gain marginal

Valeurs d'exemple

Le meilleur modèle n'est pas le plus performant : c'est le mieux proportionné.

02

Expliquer le modèle



SHAP : ouvrir la boîte noire

Pourquoi expliquer ?

1

Confiance

Le métier adhère à un modèle qu'il comprend.

2

Debug

Repérer une fuite, une feature aberrante, un biais.

3

Conformité

Justifier une décision (AI Act, audit, client).

4

Amélioration

Voir où le signal manque → retour features.

L'outil

SHAP en bref

aelion.fr

Chaque prédiction = une **valeur de base** + la somme des **contributions de chaque feature**.

L'idée

Un jeu coopératif : la part équitable de chaque feature dans la prédiction.

Global

Importance moyenne ($|SHAP|$) sur tout le jeu — quelles features comptent.

Local

Une prédiction précise expliquée — pourquoi CE machine-heure est flagué.

TreeExplainer : exact et rapide sur les modèles à base d'arbres (XGBoost, RF).

SHAP : global & local

Summary plot (global)

- 1 · chaque point = une observation.
- 2 · axe X = impact SHAP : ← vers « sain », panne →.
- 3 · couleur = valeur de la feature (basse / haute).
- 4 · features triées par impact décroissant.

Waterfall (local)

- 1 · part de la valeur de base $E[f(x)]$.
- 2 · chaque feature pousse vers la panne (rouge)...
- 3 · ... ou vers le sain (bleu).
- 4 · on arrive à la prédiction finale du cas.

Notre cas : un signal faible

1 **Importance diffuse** — beaucoup de features faibles, aucune dominante → peu de signal (AUC ~ 0,65).

2 **Explicabilité ≠ causalité** — SHAP montre ce que le modèle UTILISE, pas la cause de la panne.

3 **Une feature qui explique tout ?** — → suspecter une fuite (corrélation feature ↔ futur).

Trois décisions possibles

(1) enrichir les features (2) changer d'horizon de label (3) accepter un modèle modeste mais honnête. On choisit au TP — et on l'assume.



À vous de jouer

tp_optimisation_explicabilite.ipynb

aelion

95 chemin de Gabardie 31200 Toulouse
05 61 49 42 60 - contact@aelion.fr

aelion.fr

